

Deep Learning with PyTorch: Technical Report

Prithyadarshan | Skillset Go EduTech AI/ML Track | Week 2 (Tasks 2.1-2.4)

1. Objective and setup

This report summarises the Week 2 PyTorch work: a feed-forward network on tabular data (Task 2.1), a convolutional network on images (Task 2.2), and controlled hyperparameter experiments on the CNN (Task 2.3). Everything ran on CPU with fixed random seeds and stratified splits (70% train / 15% validation / 15% test).

- **Task 2.1:** 3-layer feed-forward network (30-64-32-1, ReLU, dropout 0.2, 4,097 parameters) on the cleaned Breast Cancer Wisconsin data from Week 1; BCEWithLogitsLoss, Adam (lr 1e-3), 40 epochs, batch 32.
- **Task 2.2:** CNN (two conv-ReLU-maxpool blocks with 16 and 32 filters, then a 64-unit dense layer with dropout 0.25, 13,706 parameters) on the 1,797-image scikit-learn handwritten-digits set (8x8 grayscale, 10 classes); cross-entropy loss, Adam, 30 epochs.
- **Task 2.3:** baseline = Adam, lr 1e-3, batch 64, 20 epochs. One factor was varied at a time (optimizer, learning rate, batch size, epochs) with the same seed and split, giving 15 runs in total. Configurations are ranked on validation accuracy; test accuracy is reported for reference.

2. Model results

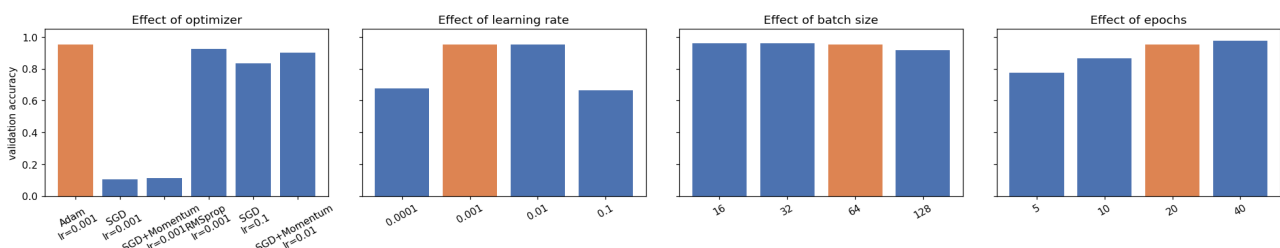
The feed-forward network reached **95.35%** test accuracy (precision 0.981, recall 0.944, F1 0.962) on an 86-sample test set. The Week 1 logistic regression scored 98.25% (F1 0.986) on a different 80/20 split with 114 test samples, so the two numbers are not a like-for-like comparison, and a gap of a few samples on sets this small is not conclusive. The dataset is close to linearly separable, so extra depth offers little here. The CNN reached **97.41%** test accuracy (loss 0.089) after 30 epochs; the weakest classes were digits 8 (recall 0.846) and 1 (precision 0.931).

3. Hyperparameter experiments

#	Factor	Optimizer	LR	Batch	Epochs	Train loss	Val acc	Test acc
1	baseline	Adam	0.001	64	20	0.158	95.2%	95.6%
2	optimizer	SGD	0.001	64	20	2.305	10.4%	10.4%
3	optimizer	SGD+Momentum	0.001	64	20	2.289	11.5%	11.8%
4	optimizer	RMSprop	0.001	64	20	0.132	92.6%	94.8%
5	optimizer	SGD	0.1	64	20	0.309	83.3%	88.1%
6	optimizer	SGD+Momentum	0.01	64	20	0.335	90.0%	91.5%
7	learning rate	Adam	0.0001	64	20	1.964	67.8%	73.3%
8	learning rate	Adam	0.01	64	20	0.013	95.2%	97.0%
9	learning rate	Adam	0.1	64	20	1.154	66.3%	68.5%
10	batch size	Adam	0.001	16	20	0.047	95.9%	97.0%
11	batch size	Adam	0.001	32	20	0.101	95.9%	96.3%
12	batch size	Adam	0.001	128	20	0.287	91.5%	92.2%
13	epochs	Adam	0.001	64	5	1.139	77.4%	83.3%
14	epochs	Adam	0.001	64	10	0.389	86.7%	90.7%
15	epochs	Adam	0.001	64	40	0.050	97.4%	98.2%

Orange row = baseline; green row = best configuration by validation accuracy.

Orange = baseline (Adam, lr=1e-3, batch=64, 20 epochs)



4. Findings

- **Optimizer.** At the shared lr of 1e-3, plain SGD and SGD with momentum did not learn at all (10.4% and 11.5% validation accuracy, training loss about 2.3, i.e. chance level for 10 classes). This reflects the learning rate rather

than the optimizer itself: with larger, SGD-appropriate rates they reached 83.3% (lr 0.1) and 90.0% (momentum, lr 0.01), still below Adam (95.2%) and RMSprop (92.6%) within 20 epochs.

- **Learning rate.** Both extremes hurt: $1e-4$ under-trains (67.8%, loss 1.96) and 0.1 is unstable (66.3%, loss 1.15). Adam at $1e-3$ and $1e-2$ both reached 95.2% validation accuracy; $1e-2$ gave the lowest training loss (0.013).
- **Batch size.** Batches of 16 and 32 scored 95.9%, 64 scored 95.2%, and 128 scored 91.5%. With epochs held fixed, smaller batches make more parameter updates, at the cost of run time (2.8s at batch 16 vs 0.8s at 128).
- **Epochs.** Validation accuracy rose steadily with training length: 77.4% (5), 86.7% (10), 95.2% (20), 97.4% (40). The model had not saturated by 20 epochs.
- **Best configuration.** Adam, lr $1e-3$, batch 64, 40 epochs: 97.4% validation and 98.2% test accuracy (baseline: 95.2% and 95.6%).

5. Limitations and conclusion

Each configuration was run once with a single seed, and the validation and test sets each hold 270 images, so one image is about 0.4 percentage points. Differences of 1-2 points (for example batch 16 vs 64) are within noise; the large effects (SGD at a small learning rate, extreme learning rates, very short training) are clear. Factors were varied one at a time, so interactions such as learning rate with batch size were not tested, and the digits images are small and easy compared with real histopathology data. Overall, learning rate and training length mattered far more than batch size or the choice between Adam and RMSprop, and Adam with a moderate learning rate was the most reliable default. Next steps are repeating runs over several seeds, a joint learning-rate and batch-size grid, and moving to larger image data with augmentation.